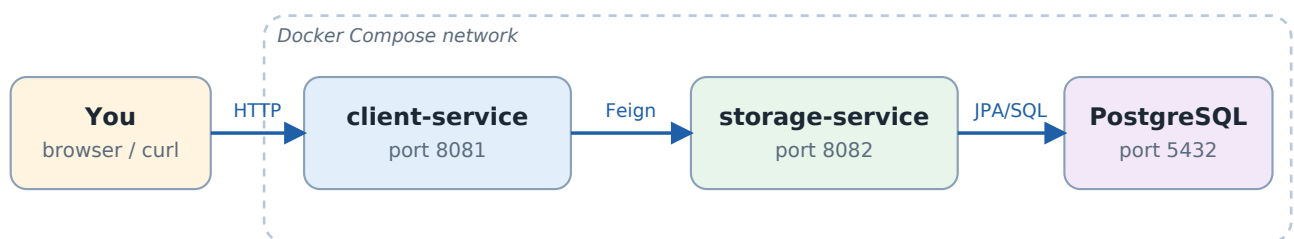


Two Microservices with Feign, Docker Desktop and PostgreSQL

A step-by-step guide for complete beginners: from installing the tools to a working system that saves data in a database



What you will use	Version in this guide
Java (JDK)	17 (any 17+ works)
Maven	3.9+
Spring Boot	3.3.5
Spring Cloud (OpenFeign)	2023.0.3
PostgreSQL (Docker image)	16
IDE	IntelliJ IDEA
Container tool	Docker Desktop with Docker Compose

Contents

1. Before You Start	4
1.1 What you are going to build	4
1.2 Words you will meet	4
1.3 How to use this guide	5
2. Installing the Tools	6
2.1 Install Java 17 (JDK)	6
2.2 Install Maven 3.9+	6
2.3 Install IntelliJ IDEA	7
2.4 Install Docker Desktop	7
2.5 Optional: a tool for sending requests	8
2.6 Check everything	9
3. Creating the Projects	10
3.1 Create a workspace folder	10
3.2 Generate storage-service with Spring Initializr	10
3.3 Generate client-service	10
3.4 Open the projects in IntelliJ	10
3.5 The final folder structure	11
4. Part 1 — storage-service	12
4.1 pom.xml	12
4.2 The main application class	13
4.3 application.yml	14
4.4 The REST controller	14
4.5 Run and test storage-service	15
5. Part 1 — client-service with Feign	16
5.1 pom.xml	16
5.2 The main application class	17
5.3 application.yml	17
5.4 The Feign client interface	18
5.5 The controller that uses Feign	19
5.6 Run both services and test	20
6. Part 1 — Running Both Services in Docker	21
6.1 How it works	21
6.2 Dockerfile for storage-service	21
6.3 Dockerfile for client-service	22
6.4 .dockerignore (both services)	22
6.5 docker-compose.yml	22

6.6 Build and start	23
7. Part 2 — Overview	24
7.1 The data flow step by step	24
7.2 Endpoints after Part 2	24
7.3 Why there are three different message classes	24
8. Part 2 — Adding PostgreSQL	25
9. Part 2 — storage-service Saves to the Database	27
9.1 Add the database dependencies	27
9.2 Database settings in application.yml	27
9.3 The entity: one row of the table	28
9.4 DTOs: the JSON shapes	29
9.5 The repository	30
9.6 The service: business logic	30
9.7 The controller for messages	31
9.8 Test storage-service alone	32
10. Part 2 — client-service Processes and Forwards	33
10.1 DTOs	33
10.2 Extend the Feign client	33
10.3 The processing service	34
10.4 The public controller	36
10.5 Quick local test (optional)	36
11. Part 2 — Everything in Docker	37
11.1 The final docker-compose.yml	37
11.2 Build and start everything	38
11.3 Send new data	38
11.4 Look inside the database	39
11.5 Two more tests	39
12. Troubleshooting	40
13. Command Cheat Sheet	40
14. Final Checklist	41
Ideas for extra practice	41

1. Before You Start

1.1 What you are going to build

You will write two small Java programs (called **microservices**) with the Spring Boot framework. They run separately, but they talk to each other over HTTP, just like two websites exchanging information. The task has two parts:

- **Part 1:** `client-service` calls `storage-service` using a **Feign client**. Both run inside Docker containers, started together with one command.
- **Part 2:** A **PostgreSQL** database is added. You send new data to `client-service`, it processes (cleans up) the data, passes it to `storage-service` through Feign, and `storage-service` saves it in the database.

Service	Port	Role
<code>client-service</code>	8081	Public API. Receives your requests and calls <code>storage-service</code> through Feign.
<code>storage-service</code>	8082	Internal service. Answers requests and, in Part 2, stores data in PostgreSQL.
<code>postgres</code> (Part 2)	5432	The database. Runs from the official PostgreSQL Docker image.

1.2 Words you will meet

Don't worry if these don't fully make sense yet. Come back to this table whenever a word confuses you.

Term	Simple explanation
Microservice	A small, independent program that does one job and talks to other programs over the network.
JDK	Java Development Kit. The tools that compile and run Java code.
Spring Boot	A Java framework that makes it quick to build web applications. It starts a built-in web server (Tomcat) for you.
Maven	A build tool. It downloads libraries ("dependencies") listed in <code>pom.xml</code> and packages your code into a <code>.jar</code> file.
REST endpoint	A URL your service answers, e.g. <code>GET http://localhost:8082/api/storage/ping</code> .
JSON	A text format for data, e.g. <code>{"author": "Alice"}</code> . Services send JSON to each other.
OpenFeign	A library that lets you call another service by writing a Java <i>interface</i> . Feign writes the HTTP code for you.
Docker image	A packaged, ready-to-run copy of a program plus everything it needs (Java, libraries).
Container	A running instance of an image. Like a lightweight, isolated mini-computer.
Docker Compose	A tool that starts several containers together, described in a <code>docker-compose.yml</code> file.
PostgreSQL	A popular open-source relational database (tables, rows, SQL).
JPA / Hibernate	Lets you save Java objects to database tables without writing SQL by hand.
Port	A number that identifies a program on a computer. Our services use 8081, 8082 and 5432.

1.3 How to use this guide

- Follow the chapters **in order**. Every chapter ends with a checkpoint that tells you what should work.
- Every code file is shown **completely**, with line numbers, followed by a table explaining each important line.
- The grey line numbers are **not** part of the code. Do not type them.
- Text in **this font** is something you type exactly: a command, file name, or value.

WATCH OUT: YAML files (`.yaml`) are sensitive to indentation. Use **spaces**, **never tabs**, and keep exactly the indentation shown. If you copy code out of a PDF, the indentation may be lost, so check it carefully or type the code yourself (typing also helps you learn).

2. Installing the Tools

You need four things: Java 17, Maven, an IDE (IntelliJ IDEA) and Docker Desktop. Follow the section for your operating system. At the end (section 2.6) you will check that everything is installed correctly.

NOTE: How to open a terminal. Windows: press the Windows key, type **PowerShell**, press Enter. macOS: press Cmd+Space, type **Terminal**, press Enter. Linux: press Ctrl+Alt+T. IntelliJ also has a built-in terminal: *View → Tool Windows → Terminal*.

2.1 Install Java 17 (JDK)

Windows

1. Open **<https://adoptium.net>** in your browser. This is Eclipse Temurin, a free, trusted build of Java.
2. Choose version **17 (LTS)**, operating system **Windows**, architecture **x64**, package **JDK**, and download the **.msi** installer.
3. Run the installer. On the *Custom Setup* screen, click the icons next to **Set JAVA_HOME variable** and **Add to PATH** and choose "*Will be installed on local hard drive*". This is important: it lets the terminal find Java.
4. Click *Next → Install → Finish*.
5. **Close and reopen** PowerShell (old windows do not see the new settings) and run **java -version**.

macOS

1. Open **<https://adoptium.net>**, choose **17 (LTS)**, **macOS**, and your chip: **aarch64** for Apple Silicon (M1/M2/M3/M4) or **x64** for Intel. Not sure? Apple menu → *About This Mac*.
2. Download the **.pkg** file, double-click it and follow the installer.
3. Alternatively, if you use Homebrew: **brew install --cask temurin@17**.
4. Open a new Terminal and run **java -version**.

Linux (Ubuntu/Debian)

```
sudo apt update
sudo apt install -y openjdk-17-jdk
java -version
```

Expected output (numbers may differ slightly, but it must say **17** or higher):

```
openjdk version "17.0.12" 2024-07-16
OpenJDK Runtime Environment Temurin-17.0.12+7 (build 17.0.12+7)
OpenJDK 64-Bit Server VM Temurin-17.0.12+7 (build 17.0.12+7, mixed mode, sharing)
```

2.2 Install Maven 3.9+

TIP: IntelliJ IDEA already contains Maven, and our Docker builds download Maven inside the container. Installing Maven yourself is still recommended so you can run **mvn** commands in any terminal.

Windows

1. Open **<https://maven.apache.org/download.cgi>** and download the **Binary zip archive** (for example **apache-maven-3.9.9-bin.zip**).

2. Create the folder `C:\tools` and extract the zip there. You now have `C:\tools\apache-maven-3.9.9` containing a `bin` folder.
3. Press the Windows key, type **environment variables**, and open "Edit the system environment variables" → button *Environment Variables...*
4. Under *User variables*, select **Path** → *Edit* → *New*, and paste `C:\tools\apache-maven-3.9.9\bin`. Click *OK* on all windows.
5. Close and reopen PowerShell and run `mvn -v`.

macOS

```
brew install maven
mvn -v
```

No Homebrew? Install it from <https://brew.sh> (one command shown on the page), or use the zip method from Windows and add the `bin` folder to your PATH in `~/ .zshrc`.

Linux

The Ubuntu package may be older than 3.9, so the easiest way is SDKMAN:

```
curl -s "https://get.sdkman.io" | bash
source "$HOME/.sdkman/bin/sdkman-init.sh"
sdk install maven
mvn -v
```

`mvn -v` must print **Apache Maven 3.9.x** and **Java version: 17....** If it shows a different Java version, your `JAVA_HOME` points to another Java; reinstall the JDK with the `Set JAVA_HOME` option.

2.3 Install IntelliJ IDEA

1. Open <https://www.jetbrains.com/idea/download>. Choose your operating system.
2. Download the free edition (*Community Edition*, or the free tier of the unified IntelliJ IDEA download if that is what the page offers). Students can also get the Ultimate edition for free at [jetbrains.com/community/education](https://www.jetbrains.com/community/education), but it is not required.
3. **Windows:** run the `.exe`. On *Installation Options*, tick *Create Desktop Shortcut*, *Add "bin" folder to the PATH* and the `.java` association. Click *Install*, then *Finish*.
4. **macOS:** open the `.dmg` and drag *IntelliJ IDEA* into *Applications*.
5. **Linux:** extract the `.tar.gz`, then run `bin/idea.sh`, or install via *Ubuntu Software* / `sudo snap install intellij-idea-community --classic`.
6. Start IntelliJ, accept the user agreement, choose a theme. You will see the *Welcome* screen with *New Project* and *Open* buttons.

TIP: Useful plugins (optional): *Settings* → *Plugins* → *Marketplace*, search **Docker** (usually already bundled). It lets you see containers inside IntelliJ.

2.4 Install Docker Desktop

Docker Desktop runs containers on your computer. It must be **running** (whale icon visible) every time you use Docker commands.

Windows 10/11 (64-bit)

1. **Enable virtualization.** Open Task Manager (Ctrl+Shift+Esc) → *Performance* → *CPU*. It should say *Virtualization: Enabled*. If it says *Disabled*, enable *Intel VT-x / AMD-V (SVM)* in your BIOS/UEFI settings (search the web for your laptop model).

2. **Install WSL 2** (Windows Subsystem for Linux). Right-click the Start button → *Terminal (Admin)* or *PowerShell (Admin)* and run `wsl --install`. Restart the computer when it finishes.
3. Open <https://www.docker.com/products/docker-desktop/> and download *Docker Desktop for Windows (AMD64)*. (Use the ARM64 version only on ARM laptops such as Snapdragon.)
4. Run **Docker Desktop Installer.exe**. Keep "**Use WSL 2 instead of Hyper-V**" ticked. Click OK and wait.
5. Click *Close and restart* (or log out and in).
6. Start **Docker Desktop** from the Start menu. Accept the service agreement. You can *skip* signing in.
7. Wait until the bottom-left corner shows **Engine running** (green).

macOS

1. On the Docker Desktop download page choose **Mac with Apple chip** or **Mac with Intel chip**.
2. Open **Docker.dmg** and drag the whale icon into *Applications*.
3. Open Docker from *Applications*, accept the agreement, allow the privileged helper when asked (enter your password).
4. Wait until the whale icon in the top menu bar stops animating and the Dashboard shows *Engine running*.

Linux (Ubuntu)

1. Follow the official instructions at <https://docs.docker.com/desktop/setup/install/linux/> (download the `.deb` and run `sudo apt install ./docker-desktop-amd64.deb`).
2. Alternatively install Docker Engine only (no GUI):
<https://docs.docker.com/engine/install/ubuntu/>. Then run `sudo usermod -aG docker $USER` and log out/in so you can use `docker` without `sudo`.

Test Docker

```
docker --version
docker compose version
docker run hello-world
```

The last command downloads a tiny test image and prints "**Hello from Docker!**". If you see that message, Docker works.

WATCH OUT: If you get "error during connect" or "Cannot connect to the Docker daemon", Docker Desktop is not running. Start it and wait for *Engine running*.

2.5 Optional: a tool for sending requests

To send POST requests with JSON in Part 2 you can use `curl` (built into Windows 10+, macOS and Linux) or **Postman** (a graphical tool, free at <https://www.postman.com/downloads/>). This guide shows commands for both terminals and describes the Postman steps.

2.6 Check everything

Command	You should see
<code>java -version</code>	version "17..." (or higher)
<code>mvn -v</code>	Apache Maven 3.9.x ... Java version: 17...
<code>docker --version</code>	Docker version 2x.x.x
<code>docker compose version</code>	Docker Compose version v2.x.x
<code>docker run hello-world</code>	Hello from Docker!

CHECKPOINT: All five commands work. You are ready to write code.

3. Creating the Projects

3.1 Create a workspace folder

Create one folder that will hold everything, for example `C:\dev\microservices-lab` on Windows or `~/dev/microservices-lab` on macOS/Linux. Avoid spaces and non-English letters in the path.

```
# Windows PowerShell
mkdir C:\dev\microservices-lab
cd C:\dev\microservices-lab

# macOS / Linux
mkdir -p ~/dev/microservices-lab
cd ~/dev/microservices-lab
```

3.2 Generate storage-service with Spring Initializr

1. Open <https://start.spring.io> in your browser.
2. **Project:** Maven. **Language:** Java. **Spring Boot:** choose a 3.3.x version if listed; otherwise pick any version (we will set 3.3.5 in `pom.xml` ourselves).
3. **Project Metadata:** Group `com.example`, Artifact `storage-service`, Name `storage-service`, Package name `com.example.storage` (change it! the default would be `com.example.storageservice`), Packaging `Jar`, Java `17`.
4. Click **ADD DEPENDENCIES** and add **Spring Web**.
5. Click **GENERATE**. A file `storage-service.zip` is downloaded.
6. Extract the zip into `microservices-lab`, so that you get `microservices-lab/storage-service/pom.xml`.

3.3 Generate client-service

1. Go back to start.spring.io (the settings stay). Change Artifact and Name to `client-service` and Package name to `com.example.client`.
2. Dependencies: **Spring Web** and **OpenFeign** (type "feign" in the search box).
3. Click **GENERATE** and extract into `microservices-lab` as well.

3.4 Open the projects in IntelliJ

1. In IntelliJ choose *File* → *Open* (or *Open* on the Welcome screen) and select the `microservices-lab` folder. Click *Trust Project* if asked.
2. Open the **Maven** tool window (the *m* icon on the right edge, or *View* → *Tool Windows* → *Maven*).
3. Click the **+** (*Add Maven Projects*) button and select `storage-service/pom.xml`. Repeat for `client-service/pom.xml`. Tip: you can also right-click each `pom.xml` in the Project view and choose *Add as Maven Project*.
4. Set Java: *File* → *Project Structure* → *Project* → *SDK*: choose **17**. If it is missing, click *Add SDK* → *JDK* and point to the folder where you installed Java (IntelliJ can also *Download JDK* for you).
5. Wait until the progress bar at the bottom finishes. Maven downloads the libraries the first time; this can take a few minutes.

NOTE: Creating packages and classes in IntelliJ. In the *Project* view open `src/main/java`. Right-click a package (for example `com.example.storage`) → *New* → *Package* and type the sub-package name (for example `controller`). Then right-click the new package → *New* → *Java Class* and type the class name. For a record choose *Record*, for an interface choose *Interface*. IntelliJ writes the `package` line for you; you then replace the file content with the code from this guide.

NOTE: application.properties vs application.yml. Spring Initializr creates `src/main/resources/application.properties`. We use the YAML format instead. Right-click that file → *Refactor* → *Rename* and rename it to `application.yml`, then replace its content. Keep only one of the two files.

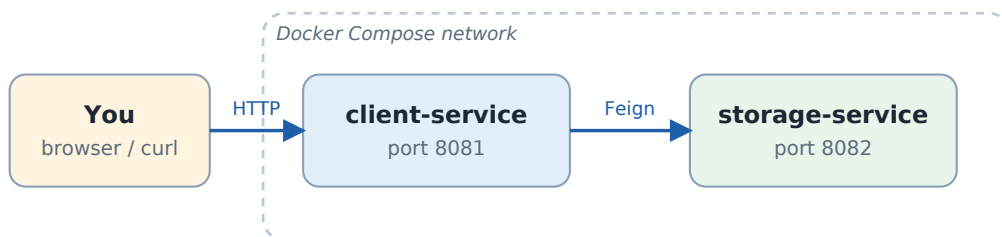
3.5 The final folder structure

This is how your workspace will look at the end of Part 2. Files marked (P2) are added in Part 2. Use it as a map whenever you are unsure where a file goes.

```
microservices-lab/
|-- docker-compose.yml
|-- storage-service/
|   |-- Dockerfile
|   |-- .dockerignore
|   |-- pom.xml
|   `-- src/main/
|       |-- java/com/example/storage/
|       |   |-- StorageServiceApplication.java
|       |   |-- controller/StorageController.java
|       |   |-- controller/MessageController.java          (P2)
|       |   |-- dto/SaveMessageRequest.java               (P2)
|       |   |-- dto/MessageResponse.java                  (P2)
|       |   |-- entity/Message.java                       (P2)
|       |   |-- repository/MessageRepository.java         (P2)
|       |   `-- service/MessageService.java               (P2)
|       `-- resources/application.yml
|-- client-service/
|   |-- Dockerfile
|   |-- .dockerignore
|   |-- pom.xml
|   `-- src/main/
|       |-- java/com/example/client/
|       |   |-- ClientServiceApplication.java
|       |   |-- controller/ClientController.java
|       |   |-- controller/MessageController.java          (P2)
|       |   |-- dto/MessageRequest.java                    (P2)
|       |   |-- dto/ProcessedMessage.java                  (P2)
|       |   |-- dto/StoredMessage.java                     (P2)
|       |   |-- feign/StorageClient.java
|       |   `-- service/MessageProcessingService.java      (P2)
|       `-- resources/application.yml
```

4. Part 1 — storage-service

We start with the service that *answers*. In Part 1 it has two simple endpoints: a health "ping" and a greeting.



4.1 pom.xml

Open `storage-service/pom.xml` and replace its **whole content** with this:

storage-service/pom.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5         https://maven.apache.org/xsd/maven-4.0.0.xsd">
6     <modelVersion>4.0.0</modelVersion>
7
8     <parent>
9         <groupId>org.springframework.boot</groupId>
10        <artifactId>spring-boot-starter-parent</artifactId>
11        <version>3.3.5</version>
12        <relativePath/>
13    </parent>
14
15    <groupId>com.example</groupId>
16    <artifactId>storage-service</artifactId>
17    <version>0.0.1-SNAPSHOT</version>
18    <name>storage-service</name>
19
20    <properties>
21        <java.version>17</java.version>
22    </properties>
23
24    <dependencies>
25        <dependency>
26            <groupId>org.springframework.boot</groupId>
27            <artifactId>spring-boot-starter-web</artifactId>
28        </dependency>
29
30        <dependency>
31            <groupId>org.springframework.boot</groupId>
32            <artifactId>spring-boot-starter-test</artifactId>
33            <scope>test</scope>
34        </dependency>
35    </dependencies>
36
37    <build>
38        <plugins>
39            <plugin>
40                <groupId>org.springframework.boot</groupId>
41                <artifactId>spring-boot-maven-plugin</artifactId>
42            </plugin>
43        </plugins>
44    </build>
45 </project>
```

Line	What it means
1	Standard XML header. Every <code>pom.xml</code> starts like this.
2–5	The root element. The attributes tell Maven (and IntelliJ) which POM format is used, so it can check the file.
6	Always <code>4.0.0</code> for Maven 2 and later.
8–13	Inherit settings from spring-boot-starter-parent version 3.3.5 . It chooses matching versions for all Spring libraries, so we do not write versions for them. <code><relativePath/></code> means: download the parent from the internet, not from a local folder.
15–18	The identity of <i>our</i> project: organization (<code>groupId</code>), project name (<code>artifactId</code>) and version. SNAPSHOT means "work in progress".
20–22	Compile for Java 17 .
27	The most important dependency: Spring MVC (for REST controllers), JSON support (Jackson) and the embedded Tomcat web server.
32–33	Testing libraries (JUnit etc.). <code>scope test</code> means they are only used for tests, not included in the final app.
37–44	The spring-boot-maven-plugin packages the application as one runnable "fat" <code>.jar</code> containing all libraries. Docker will run this jar.

TIP: After editing a `pom.xml`, IntelliJ shows a small Maven icon (*Load Maven Changes*) in the top-right of the editor. Click it (or press `Ctrl+Shift+O` / `Cmd+Shift+I`) so the new dependencies are downloaded.

4.2 The main application class

Spring Initializr already created this file at

`src/main/java/com/example/storage/StorageServiceApplication.java`. Check that it looks exactly like this:

StorageServiceApplication.java

```

1 package com.example.storage;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class StorageServiceApplication {
8
9     public static void main(String[] args) {
10         SpringApplication.run(StorageServiceApplication.class, args);
11     }
12 }
```

Line	What it means
1	The package (folder) this class belongs to. It must match the folder path <code>com/example/storage</code> .
3–4	Bring in the two Spring classes we use below.
6	Turns on Spring Boot: auto-configuration (sets up Tomcat, JSON, etc.) and component scanning (finds our controllers in this package and all sub-packages, like <code>controller</code>).
7	A normal Java class. The name must match the file name.
9	The entry point: Java starts the program here.
10	Starts Spring and the web server. The app keeps running until you stop it.

4.3 application.yml

Rename `src/main/resources/application.properties` to `application.yml` (see section 3.4) and write:

`storage-service/src/main/resources/application.yml`

```
1 server:
2   port: 8082
3
4 spring:
5   application:
6     name: storage-service
```

Line	What it means
1–2	The web server listens on port 8082 (the default would be 8080). Note the two spaces before port .
4–6	A name for the application. It appears in logs and is good practice in microservices.

4.4 The REST controller

Create the package `com.example.storage.controller` and inside it the class `StorageController`:

`storage-service/.../controller/StorageController.java`

```
1 package com.example.storage.controller;
2
3 import org.springframework.web.bind.annotation.GetMapping;
4 import org.springframework.web.bind.annotation.PathVariable;
5 import org.springframework.web.bind.annotation.RequestMapping;
6 import org.springframework.web.bind.annotation.RestController;
7
8 import java.time.LocalDateTime;
9 import java.util.Map;
10
11 @RestController
12 @RequestMapping("/api/storage")
13 public class StorageController {
14
15     @GetMapping("/ping")
16     public Map<String, Object> ping() {
17         return Map.of(
18             "service", "storage-service",
19             "status", "UP",
20             "time", LocalDateTime.now().toString()
21         );
22     }
23
24     @GetMapping("/greet/{name}")
25     public String greet(@PathVariable("name") String name) {
26         return "Hello, " + name + "! Greetings from storage-service.";
27     }
28 }
```

Line	What it means
1	This class lives in the controller sub-package.
3–6	Import the Spring Web annotations we use.
8–9	Java classes for the current time and for a key-value map.
11	Tells Spring: this class handles HTTP requests and the return values are sent back as the response body (objects become JSON automatically).
12	All URLs in this class start with /api/storage .

Line	What it means
15	This method answers GET requests to <code>/api/storage/ping</code> .
16	Returns a map. Spring converts it to JSON like <code>{"service":"storage-service",...}</code> .
17–21	<code>Map.of</code> builds an unchangeable map with three key-value pairs, including the current time as text.
24	Answers GET <code>/api/storage/greet/Anna</code> . The part in curly braces <code>{name}</code> is a path variable .
25	Copies the <code>{name}</code> part of the URL into the Java parameter <code>name</code> .
26	Returns plain text, e.g. <code>Hello, Anna! Greetings from storage-service</code> .

4.5 Run and test storage-service

1. Open `StorageServiceApplication.java` and click the green ► triangle next to `public class` → `Run 'StorageServiceApplication'`.
2. Watch the `Run` window at the bottom. After a few seconds you should see a line like `Tomcat started on port 8082 (http)` and `Started StorageServiceApplication`.
3. Open your browser at `http://localhost:8082/api/storage/ping`.
4. Then try `http://localhost:8082/api/storage/greet/Anna`.

Expected result of the ping:

```
{"service":"storage-service","status":"UP","time":"2026-09-23T10:15:30.123"}
```

CHECKPOINT: Both URLs answer in the browser. **Leave storage-service running** for the next chapter.

WATCH OUT: *"Port 8082 was already in use"* means an old copy is still running. Stop it with the red square ■ in the `Run` window, or change the port in `application.yml`.

5. Part 1 — client-service with Feign

Now we build the service that *asks*. When you call `client-service`, it uses a Feign client to call `storage-service` and returns the answer to you.

5.1 pom.xml

Replace the whole content of `client-service/pom.xml`:

`client-service/pom.xml`

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5                              https://maven.apache.org/xsd/maven-4.0.0.xsd">
6      <modelVersion>4.0.0</modelVersion>
7
8      <parent>
9          <groupId>org.springframework.boot</groupId>
10         <artifactId>spring-boot-starter-parent</artifactId>
11         <version>3.3.5</version>
12         <relativePath/>
13     </parent>
14
15     <groupId>com.example</groupId>
16     <artifactId>client-service</artifactId>
17     <version>0.0.1-SNAPSHOT</version>
18     <name>client-service</name>
19
20     <properties>
21         <java.version>17</java.version>
22         <spring-cloud.version>2023.0.3</spring-cloud.version>
23     </properties>
24
25     <dependencies>
26         <dependency>
27             <groupId>org.springframework.boot</groupId>
28             <artifactId>spring-boot-starter-web</artifactId>
29         </dependency>
30
31         <dependency>
32             <groupId>org.springframework.cloud</groupId>
33             <artifactId>spring-cloud-starter-openfeign</artifactId>
34         </dependency>
35
36         <dependency>
37             <groupId>org.springframework.boot</groupId>
38             <artifactId>spring-boot-starter-test</artifactId>
39             <scope>test</scope>
40         </dependency>
41     </dependencies>
42
43     <dependencyManagement>
44         <dependencies>
45             <dependency>
46                 <groupId>org.springframework.cloud</groupId>
47                 <artifactId>spring-cloud-dependencies</artifactId>
48                 <version>${spring-cloud.version}</version>
49                 <type>pom</type>
50                 <scope>import</scope>
51             </dependency>
52         </dependencies>
53     </dependencyManagement>
54
55     <build>
56         <plugins>
57             <plugin>
```

```

58         <groupId>org.springframework.boot</groupId>
59         <artifactId>spring-boot-maven-plugin</artifactId>
60     </plugin>
61 </plugins>
62 </build>
63 </project>

```

Line	What it means
8–13	Same Spring Boot parent 3.3.5 as storage-service.
16	The name of this project.
22	Spring Cloud release train 2023.0.3 ("Leyton"), which is the one compatible with Spring Boot 3.3. We define it once and use it below.
28	Web server and REST controllers, as before.
33	OpenFeign : lets us call other services through a Java interface. No version here, it comes from the BOM below.
43–53	Imports the Spring Cloud BOM (Bill of Materials): a list of matching versions for all Spring Cloud libraries. type pom + scope import means "take the versions from that list".
55–62	Build a runnable jar, as before.

WATCH OUT: Spring Boot and Spring Cloud versions must match. Spring Boot 3.3.x works with Spring Cloud 2023.0.x. A mismatch usually causes an error at startup mentioning *"Spring Boot [x] is not compatible"*.

5.2 The main application class

Open `ClientServiceApplication.java` and add the `@EnableFeignClients` annotation and its import:

ClientServiceApplication.java

```

1  package com.example.client;
2
3  import org.springframework.boot.SpringApplication;
4  import org.springframework.boot.autoconfigure.SpringBootApplication;
5  import org.springframework.cloud.openfeign.EnableFeignClients;
6
7  @SpringBootApplication
8  @EnableFeignClients
9  public class ClientServiceApplication {
10
11      public static void main(String[] args) {
12          SpringApplication.run(ClientServiceApplication.class, args);
13      }
14  }

```

Line	What it means
5	Import the Feign annotation.
7	Starts Spring Boot, same as in storage-service.
8	Tells Spring to search for interfaces annotated with <code>@FeignClient</code> and to create working HTTP clients from them automatically. Without this line, Feign clients are not created and the app fails with <i>"No qualifying bean of type StorageClient"</i> .
12	Start the application.

5.3 application.yml

`client-service/src/main/resources/application.yml`

```

1  server:
2    port: 8081
3
4  spring:
5    application:
6      name: client-service
7    cloud:
8      openfeign:
9        client:
10         config:
11           default:
12             connect-timeout: 5000
13             read-timeout: 5000
14
15  storage:
16    service:
17      url: ${STORAGE_SERVICE_URL:http://localhost:8082}

```

Line	What it means
2	client-service listens on port 8081 .
6	The application name.
7–13	Feign settings for all clients (default): wait at most 5000 ms (5 s) to connect and 5 s for an answer. Without limits, a request could hang for a long time if storage-service is down.
15–17	Our own setting: the address of storage-service. <code>\${STORAGE_SERVICE_URL:http://localhost:8082}</code> means: use the environment variable <code>STORAGE_SERVICE_URL</code> if it exists; otherwise use <code>http://localhost:8082</code> . Locally we use the default; in Docker we set the variable (chapter 6).

5.4 The Feign client interface

Create the package `com.example.client.feign` and inside it an **interface** (not a class) named `StorageClient`:

`client-service/.../feign/StorageClient.java`

```

1  package com.example.client.feign;
2
3  import org.springframework.cloud.openfeign.FeignClient;
4  import org.springframework.web.bind.annotation.GetMapping;
5  import org.springframework.web.bind.annotation.PathVariable;
6
7  import java.util.Map;
8
9  @FeignClient(name = "storage-service", url = "${storage.service.url}")
10 public interface StorageClient {
11
12     @GetMapping("/api/storage/ping")
13     Map<String, Object> ping();
14
15     @GetMapping("/api/storage/greet/{name}")
16     String greet(@PathVariable("name") String name);
17 }

```

Line	What it means
3	The Feign annotation.
4–5	We reuse the same Spring Web annotations as in a controller. Feign understands them.
9	Makes this interface a Feign client. <code>name</code> is a label for the client. <code>url</code> is where requests are sent; <code>\${storage.service.url}</code> reads the value from <code>application.yml</code> .
10	Only an interface: we don't write any implementation . Spring + Feign create the implementation at startup.

Line	What it means
12–13	Calling <code>storageClient.ping()</code> sends <code>GET {url}/api/storage/ping</code> and converts the JSON answer into a <code>Map</code> . The path must match storage-service's controller exactly.
15–16	Calling <code>greet("Anna")</code> sends <code>GET {url}/api/storage/greet/Anna</code> and returns the text answer. In Feign interfaces always write the name inside <code>@PathVariable("name")</code> .

5.5 The controller that uses Feign

Create package `com.example.client.controller` and class `ClientController`:

`client-service/.../controller/ClientController.java`

```

1  package com.example.client.controller;
2
3  import com.example.client.feign.StorageClient;
4  import org.springframework.web.bind.annotation.GetMapping;
5  import org.springframework.web.bind.annotation.PathVariable;
6  import org.springframework.web.bind.annotation.RequestMapping;
7  import org.springframework.web.bind.annotation.RestController;
8
9  import java.util.Map;
10
11 @RestController
12 @RequestMapping("/api/client")
13 public class ClientController {
14
15     private final StorageClient storageClient;
16
17     public ClientController(StorageClient storageClient) {
18         this.storageClient = storageClient;
19     }
20
21     @GetMapping("/ping-storage")
22     public Map<String, Object> pingStorage() {
23         Map<String, Object> storageAnswer = storageClient.ping();
24         return Map.of(
25             "service", "client-service",
26             "storageAnswer", storageAnswer
27         );
28     }
29
30     @GetMapping("/greet/{name}")
31     public String greet(@PathVariable("name") String name) {
32         return storageClient.greet(name);
33     }
34 }

```

Line	What it means
3	Import our Feign interface.
11	This class answers HTTP requests with JSON/text.
12	All URLs here start with <code>/api/client</code> .
15	A field that will hold the Feign client. <code>final</code> = assigned once.
17–19	Constructor injection: Spring sees the constructor needs a <code>StorageClient</code> , and passes in the one Feign created. You never write <code>new StorageClient()</code> .
21	Answers <code>GET /api/client/ping-storage</code> .
23	This line is the microservice call. It looks like a normal method call, but Feign sends an HTTP request to storage-service and waits for the answer.
24–27	Wrap the answer: our own service name plus what storage-service replied.
30–32	Passes the name on to storage-service and returns its greeting.

5.6 Run both services and test

1. Make sure **storage-service** is still running (from chapter 4).
2. Open `ClientServiceApplication.java` and click the green ► → *Run*. Wait for **Tomcat** started on port 8081.
3. Open **`http://localhost:8081/api/client/ping-storage`** in the browser.
4. Open **`http://localhost:8081/api/client/greet/Anna`**.

Expected result of the first URL:

```
{"service":"client-service","storageAnswer":{"service":"storage-service","status":"UP",
"time":"2026-09-23T10:20:01.456"}}
```

TIP: Try an experiment: stop storage-service (red square) and reload the client URL. You get an error (HTTP 500, and a **Connection refused** message in the client log). This proves the client really calls the other service. Start storage-service again afterwards.

CHECKPOINT: Two services talk to each other through Feign on your computer. **Stop both** in IntelliJ now (red square), because in the next chapter Docker will use the same ports.

6. Part 1 — Running Both Services in Docker

6.1 How it works

A **Dockerfile** is a recipe for building an image of one service. **docker-compose.yml** describes which containers to start and how they connect. Docker Compose puts all containers into one private network, where each container can reach the others **by service name**.

WATCH OUT: The most common beginner mistake: inside a container, `localhost` means *that container itself*, not your computer and not the other container. That is why, in Docker, client-service must call `http://storage-service:8082` (the service name), not `http://localhost:8082`. Our `STORAGE_SERVICE_URL` variable handles exactly this.

6.2 Dockerfile for storage-service

In IntelliJ right-click the `storage-service` folder → *New* → *File*, name it exactly **Dockerfile** (capital D, no extension) and write:

storage-service/Dockerfile

```
1 # ----- Stage 1: build the JAR with Maven -----
2 FROM maven:3.9-eclipse-temurin-17 AS build
3 WORKDIR /app
4 COPY pom.xml .
5 RUN mvn -B dependency:go-offline
6 COPY src ./src
7 RUN mvn -B clean package -DskipTests
8
9 # ----- Stage 2: run the JAR with a small Java runtime -----
10 FROM eclipse-temurin:17-jre
11 WORKDIR /app
12 COPY --from=build /app/target/*.jar app.jar
13 EXPOSE 8082
14 ENTRYPOINT ["java", "-jar", "app.jar"]
```

Line	What it means
1	A comment. This Dockerfile has two stages : first build, then run. This is called a <i>multi-stage build</i> .
2	Start from an official image that already has Maven 3.9 and JDK 17. We call this stage build . You don't need Maven on your PC for this.
3	Inside the image, work in the folder <code>/app</code> (created automatically).
4	Copy only <code>pom.xml</code> first into <code>/app</code> .
5	Download all dependencies. Because this happens before copying the code, Docker caches this step: when you change only Java code, the next build is much faster. <code>-B</code> = batch mode (cleaner log).
6	Now copy the source code.
7	Compile and package the app into <code>target/storage-service-0.0.1-SNAPSHOT.jar</code> . <code>-DskipTests</code> skips tests (in Part 2 the default test would need a database).
10	Stage 2 starts from a small image with only the Java runtime (JRE). Maven and the source code are not included, so the final image is smaller.
11	Working folder in the final image.
12	Copy the jar from the build stage and rename it to <code>app.jar</code> .
13	Documents that the app listens on port 8082 inside the container.
14	The command run when the container starts: <code>java -jar app.jar</code> .

6.3 Dockerfile for client-service

Create `client-service/Dockerfile` with **exactly the same content**, except one line: the port.

```
EXPOSE 8081
```

6.4 .dockerignore (both services)

Create a file named `.dockerignore` in **both** `storage-service` and `client-service` folders:

.dockerignore

```
1 target/
2 .idea/
3 *.iml
4 .mvn/
5 mvnw
6 mvnw.cmd
7 HELP.md
```

These files and folders are **not** sent to Docker when building. Old build output (`target/`) and IDE files are not needed and would only slow the build down.

6.5 docker-compose.yml

Create `docker-compose.yml` in the **root** folder `microservices-lab` (next to the two service folders):

microservices-lab/docker-compose.yml

```
1 services:
2   storage-service:
3     build: ./storage-service
4     container_name: storage-service
5     ports:
6       - "8082:8082"
7
8   client-service:
9     build: ./client-service
10    container_name: client-service
11    ports:
12      - "8081:8081"
13    environment:
14      STORAGE_SERVICE_URL: http://storage-service:8082
15    depends_on:
16      - storage-service
```

Line	What it means
1	The list of containers to start. (Older tutorials start with <code>version: "3"</code> ; this line is no longer needed.)
2	The service name . It is also the host name other containers use to reach it.
3	Build the image from the <code>Dockerfile</code> in the <code>storage-service</code> folder.
4	A readable name for the container in Docker Desktop.
5–6	Publish the port: <code>HOST:CONTAINER</code> . Your computer's port 8082 forwards to port 8082 inside the container, so you can open it in the browser.
8	The second service.
9	Build from the client-service Dockerfile.
12	Make client-service reachable at <code>http://localhost:8081</code> from your computer.
13–14	Set the environment variable. Our <code>application.yml</code> reads it, so Feign calls <code>http://storage-service:8082</code> , the service name in the Docker network.

Line	What it means
15–16	Start storage-service before client-service.

6.6 Build and start

1. Make sure Docker Desktop is running (*Engine running*) and both services are stopped in IntelliJ.
2. Open a terminal **in the `microservices-lab` folder** (in IntelliJ: *View* → *Tool Windows* → *Terminal*; it opens in the project folder).
3. Run the command below. The first build takes several minutes because images and dependencies are downloaded.

```
docker compose up --build
```

You will see the logs of both containers mixed, with prefixes `storage-service |` and `client-service |`. Wait until both print `Started ...Application`. Then open in the browser:

- `http://localhost:8082/api/storage/ping` (storage-service directly)
- `http://localhost:8081/api/client/ping-storage` (client → Feign → storage, inside Docker)
- `http://localhost:8081/api/client/greet/Docker`

Useful Docker commands

Command	What it does
<code>docker compose up --build</code>	Build images (again) and start all containers, logs in the terminal. Stop with Ctrl+C .
<code>docker compose up --build -d</code>	Same, but in the background (<i>detached</i>). The terminal stays free.
<code>docker compose ps</code>	Show running containers of this project and their ports.
<code>docker compose logs -f client-service</code>	Follow the logs of one service (Ctrl+C to stop following).
<code>docker compose down</code>	Stop and remove the containers (images stay).
<code>docker compose down -v</code>	Also delete volumes, i.e. database data in Part 2.

TIP: Changed your Java code? Always rebuild: `docker compose up --build`. Without `--build`, Docker runs the old image and you will wonder why your change doesn't work.

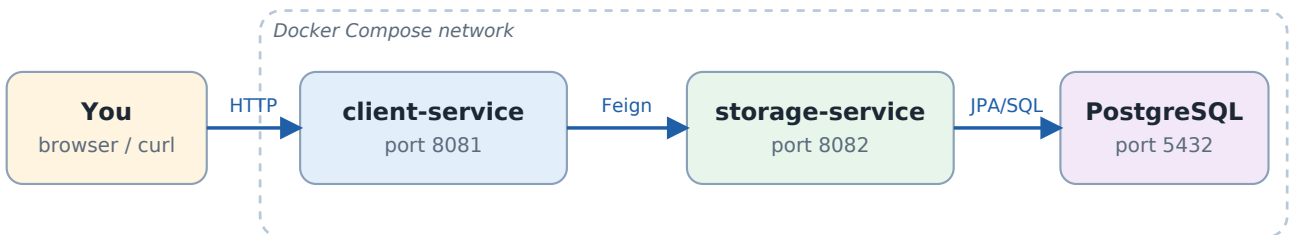
Look at it in Docker Desktop

Open Docker Desktop → **Containers**. You see a group `microservices-lab` with two containers. Click a container to see its **Logs**, its **Inspect** details, and an **Exec** tab (a terminal inside the container). The **Images** tab lists the images Compose built. The stop/start buttons do the same as the commands above.

CHECKPOINT: Part 1 done. Both services run in Docker and communicate through Feign using the service name. Stop them with Ctrl+C (or `docker compose down`) before continuing.

7. Part 2 — Overview

Now we add a PostgreSQL database and a real data flow. You send a **message** (an author and a text) to client-service. It checks and cleans the data, then sends it through Feign to storage-service, which saves it in PostgreSQL and returns the saved record.



7.1 The data flow step by step

1. You send `POST http://localhost:8081/api/messages` with JSON `{"author": " aLiCe ", "text": " hello microservices world "}`.
2. **client-service processes the data:** checks that author and text are not empty, trims spaces, turns `aLiCe` into `Alice`, replaces multiple spaces with one, counts the words (3) and adds the time of processing.
3. client-service calls `storageClient.sendMessage(...)`. Feign sends `POST http://storage-service:8082/api/storage/messages` with the processed JSON.
4. **storage-service** converts the JSON into a `Message` entity and saves it with JPA. PostgreSQL gives it an `id`; storage-service adds `savedAt`.
5. The saved record goes back: PostgreSQL → storage-service → Feign → client-service → you.

7.2 Endpoints after Part 2

Service	Method + URL	Purpose
client-service	<code>POST /api/messages</code>	Public: receive, process and store a new message.
client-service	<code>GET /api/messages</code>	Public: list all stored messages (via Feign).
storage-service	<code>POST /api/storage/messages</code>	Internal: save a processed message to PostgreSQL.
storage-service	<code>GET /api/storage/messages</code>	Internal: read all messages from PostgreSQL.
both	Part 1 endpoints	Still work unchanged.

7.3 Why there are three different message classes

Each step of the flow has its own data shape. Keeping them separate is a good habit: the public API can change without breaking the database, and the database entity is never exposed directly.

Class	Where	Contains
<code>MessageRequest</code>	client (input)	What the user sends: <code>author</code> , <code>text</code> (raw, uncleaned).
<code>ProcessedMessage</code> / <code>SaveMessageRequest</code>	client → storage	Cleaned data: <code>author</code> , <code>text</code> , <code>wordCount</code> , <code>processedAt</code> . Two names for the same JSON, one in each service.
<code>Message</code>	storage (database)	The JPA entity = one row in table <code>messages</code> .

Class	Where	Contains
<code>MessageResponse</code> / <code>StoredMessage</code>	storage → client → user	The saved record including <code>id</code> and <code>savedAt</code> .

NOTE: Feign and Spring convert objects to JSON and back **by field name**. The field names in `ProcessedMessage` (client) and `SaveMessageRequest` (storage) must be identical, even though the class names differ.

8. Part 2 — Adding PostgreSQL

First we start only the database, so that we can develop and test storage-service from IntelliJ. Open `docker-compose.yml` and add the `postgres` service at the top of `services:` and the `volumes:` section at the very bottom. The complete final file is shown in chapter 11; the new part is:

docker-compose.yml (new parts)

```

1  services:
2    postgres:
3      image: postgres:16
4      container_name: postgres-db
5      environment:
6        POSTGRES_DB: storagedb
7        POSTGRES_USER: storage_user
8        POSTGRES_PASSWORD: storage_pass
9      ports:
10     - "5432:5432"
11     volumes:
12     - postgres-data:/var/lib/postgresql/data
13     healthcheck:
14       test: ["CMD-SHELL", "pg_isready -U storage_user -d storagedb"]
15       interval: 5s
16       timeout: 5s
17       retries: 10
18
19   # ... storage-service and client-service stay as before ...
20
21  volumes:
22    postgres-data:

```

Line	What it means
2	Service name <code>postgres</code> . Other containers reach the database at host <code>postgres</code> .
3	Use the official PostgreSQL 16 image from Docker Hub. Nothing to build.
4	Name shown in Docker Desktop; we also use it in <code>docker exec</code> later.
5–8	On first start, the image creates the database <code>storagedb</code> and the user <code>storage_user</code> with password <code>storage_pass</code> . (Use strong passwords in real projects!)
10	Publish the database port so storage-service running in IntelliJ (outside Docker) can reach it at <code>localhost:5432</code> .
11–12	Store the database files in a named Docker volume . Data survives when the container is stopped or removed.
13–17	Every 5 s Docker runs <code>pg_isready</code> to check if the database accepts connections. The container becomes <i>healthy</i> when it does. storage-service will wait for this.

Start only the database in the background:

```

docker compose up -d postgres
docker compose ps

```

After a few seconds `docker compose ps` shows `postgres-db` with status **(healthy)**.

WATCH OUT: If PostgreSQL is already installed on your computer, port 5432 is taken and you see "*port is already allocated*". Change the mapping to "`5433:5432`" and later start storage-service locally with the environment variable `DB_PORT=5433` (in IntelliJ: *Run* → *Edit Configurations* → *Environment variables*).

9. Part 2 — storage-service Saves to the Database

9.1 Add the database dependencies

In `storage-service/pom.xml` replace the `<dependencies>` block with this one (the rest of the file stays the same). Then click *Load Maven Changes*.

`storage-service/pom.xml` (`<dependencies>` section)

```
1  <dependencies>
2    <dependency>
3      <groupId>org.springframework.boot</groupId>
4      <artifactId>spring-boot-starter-web</artifactId>
5    </dependency>
6
7    <!-- NEW in Part 2: JPA (talk to the database with Java objects) -->
8    <dependency>
9      <groupId>org.springframework.boot</groupId>
10     <artifactId>spring-boot-starter-data-jpa</artifactId>
11   </dependency>
12
13   <!-- NEW in Part 2: the PostgreSQL JDBC driver -->
14   <dependency>
15     <groupId>org.postgresql</groupId>
16     <artifactId>postgresql</artifactId>
17     <scope>runtime</scope>
18   </dependency>
19
20   <dependency>
21     <groupId>org.springframework.boot</groupId>
22     <artifactId>spring-boot-starter-test</artifactId>
23     <scope>test</scope>
24   </dependency>
25 </dependencies>
```

Line	What it means
10	Spring Data JPA with Hibernate : maps Java classes to tables and gives us ready-made repositories (save, find, delete) without SQL.
15–17	The JDBC driver that knows how to talk to PostgreSQL. <code>runtime</code> = needed only when the app runs, not for compiling.

WATCH OUT: From now on storage-service **needs a running database to start**. Without it you get *"Connection to localhost:5432 refused"*. Keep the postgres container running.

9.2 Database settings in application.yml

`storage-service/src/main/resources/application.yml`

```
1  server:
2    port: 8082
3
4  spring:
5    application:
6      name: storage-service
7    datasource:
8      url: jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:storagedb}
9      username: ${DB_USER:storage_user}
10     password: ${DB_PASSWORD:storage_pass}
11   jpa:
12     hibernate:
13       ddl-auto: update
```

```
14 show-sql: true
15 open-in-view: false
```

Line	What it means
2	Unchanged.
7	Settings for the database connection.
8	The JDBC address: <code>jdbc:postgresql://HOST:PORT/DATABASE</code> . Each part has a variable with a default: locally it becomes <code>localhost:5432/storagedb</code> ; in Docker we set <code>DB_HOST=postgres</code> .
9–10	Login data, the same as in <code>docker-compose.yml</code> .
13	Hibernate creates or updates the table automatically from the entity class. Very handy for learning. (In production, teams use migration tools like Flyway instead.)
14	Print every SQL statement in the log, so you can see what Hibernate does.
15	Close the database session when the service method ends. Recommended setting; it also removes a warning from the log.

9.3 The entity: one row of the table

Create package `com.example.storage.entity` and class `Message`:

`storage-service/.../entity/Message.java`

```
1 package com.example.storage.entity;
2
3 import jakarta.persistence.Column;
4 import jakarta.persistence.Entity;
5 import jakarta.persistence.GeneratedValue;
6 import jakarta.persistence.GenerationType;
7 import jakarta.persistence.Id;
8 import jakarta.persistence.PrePersist;
9 import jakarta.persistence.Table;
10
11 import java.time.LocalDateTime;
12
13 @Entity
14 @Table(name = "messages")
15 public class Message {
16
17     @Id
18     @GeneratedValue(strategy = GenerationType.IDENTITY)
19     private Long id;
20
21     @Column(nullable = false, length = 100)
22     private String author;
23
24     @Column(nullable = false, length = 1000)
25     private String text;
26
27     @Column(name = "word_count", nullable = false)
28     private int wordCount;
29
30     @Column(name = "processed_at", nullable = false)
31     private LocalDateTime processedAt;
32
33     @Column(name = "saved_at", nullable = false)
34     private LocalDateTime savedAt;
35
36     protected Message() {
37         // required by JPA
38     }
39
40     public Message(String author, String text, int wordCount, LocalDateTime processedAt) {
41         this.author = author;
42         this.text = text;
```

```

43     this.wordCount = wordCount;
44     this.processedAt = processedAt;
45 }
46
47 @PrePersist
48 void onSave() {
49     this.savedAt = LocalDateTime.now();
50 }
51
52 public Long getId() { return id; }
53 public String getAuthor() { return author; }
54 public String getText() { return text; }
55 public int getWordCount() { return wordCount; }
56 public LocalDateTime getProcessedAt() { return processedAt; }
57 public LocalDateTime getSavedAt() { return savedAt; }
58 }

```

Line	What it means
3–9	JPA annotations. In Spring Boot 3 they are in <code>jakarta.persistence</code> (old tutorials use <code>javax.persistence</code> , which no longer works).
13	This class is stored in the database. Hibernate manages it.
14	The table is called <code>messages</code> .
17	The next field is the primary key (unique ID of each row).
18	The database generates the ID automatically (1, 2, 3...). <code>IDENTITY</code> uses PostgreSQL's auto-increment column.
19	The ID field. <code>Long</code> (object) instead of <code>long</code> , because it is <code>null</code> before saving.
21–22	Column <code>author</code> , required (NOT NULL), max 100 characters.
24–25	Column <code>text</code> , required, max 1000 characters.
27–28	Column name <code>word_count</code> (SQL style) for the Java field <code>wordCount</code> .
30–31	When client-service processed the message.
33–34	When storage-service saved it.
36–38	An empty constructor. JPA requires it to create objects when reading rows from the database. <code>protected</code> so our own code doesn't use it by accident.
40–45	The constructor we use to create a new message. <code>id</code> and <code>savedAt</code> are not parameters because they are filled automatically.
47–50	Hibernate calls <code>onSave()</code> right before inserting the row, so <code>savedAt</code> is always set.
52–57	Getters: read-only access to the fields. We need them to build the response.

9.4 DTOs: the JSON shapes

Create package `com.example.storage.dto`. A **record** (Java 16+) is a short way to write a class that only carries data: Java generates the constructor, getters (`author()`), `equals` and `toString` for you. In IntelliJ choose *New* → *Java Class* → *Record*.

`storage-service/.../dto/SaveMessageRequest.java`

```

1 package com.example.storage.dto;
2
3 import java.time.LocalDateTime;
4
5 public record SaveMessageRequest(
6     String author,
7     String text,
8     int wordCount,
9     LocalDateTime processedAt
10 ) {
11 }

```

Line	What it means
5–10	The JSON storage-service receives . Spring converts <code>{"author": "...", "text": "...", "wordCount": 3, "processedAt": "2026-..."}</code> into this record automatically. Field names must match the JSON keys.

storage-service/.../dto/MessageResponse.java

```

1 package com.example.storage.dto;
2
3 import com.example.storage.entity.Message;
4
5 import java.time.LocalDateTime;
6
7 public record MessageResponse(
8     Long id,
9     String author,
10    String text,
11    int wordCount,
12    LocalDateTime processedAt,
13    LocalDateTime savedAt
14 ) {
15     public static MessageResponse from(Message m) {
16         return new MessageResponse(m.getId(), m.getAuthor(), m.getText(),
17             m.getWordCount(), m.getProcessedAt(), m.getSavedAt());
18     }
19 }

```

Line	What it means
3	We convert from the entity, so we import it.
7–14	The JSON storage-service returns : all fields including <code>id</code> and <code>savedAt</code> .
15–18	A small helper ("factory method") that copies the values from an entity into a response. <code>static</code> means you call it as <code>MessageResponse.from(message)</code> .

9.5 The repository

Create package `com.example.storage.repository` and an **interface** `MessageRepository`:

storage-service/.../repository/MessageRepository.java

```

1 package com.example.storage.repository;
2
3 import com.example.storage.entity.Message;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface MessageRepository extends JpaRepository<Message, Long> {
7 }

```

Line	What it means
4	The Spring Data base interface.
6	That's all! By extending <code>JpaRepository</code> for entity <code>Message</code> with ID type <code>Long</code> , Spring creates an implementation at startup with methods like <code>save()</code> , <code>findAll()</code> , <code>findById()</code> , <code>deleteById()</code> and <code>count()</code> . No SQL needed.

9.6 The service: business logic

Create package `com.example.storage.service` and class `MessageService`:

storage-service/.../service/MessageService.java

```

1 package com.example.storage.service;
2

```

```

3 import com.example.storage.dto.MessageResponse;
4 import com.example.storage.dto.SaveMessageRequest;
5 import com.example.storage.entity.Message;
6 import com.example.storage.repository.MessageRepository;
7 import org.springframework.stereotype.Service;
8 import org.springframework.transaction.annotation.Transactional;
9
10 import java.util.List;
11
12 @Service
13 public class MessageService {
14
15     private final MessageRepository repository;
16
17     public MessageService(MessageRepository repository) {
18         this.repository = repository;
19     }
20
21     @Transactional
22     public MessageResponse save(SaveMessageRequest request) {
23         Message message = new Message(
24             request.author(),
25             request.text(),
26             request.wordCount(),
27             request.processedAt()
28         );
29         Message saved = repository.save(message);
30         return MessageResponse.from(saved);
31     }
32
33     @Transactional(readOnly = true)
34     public List<MessageResponse> findAll() {
35         return repository.findAll().stream()
36             .map(MessageResponse::from)
37             .toList();
38     }
39 }

```

Line	What it means
12	Marks a service class (business logic). Spring creates one instance and can inject it into controllers.
15–19	Constructor injection of the repository, same pattern as in the Feign controller.
21	Runs the method in a database transaction : either everything is saved, or (on an error) nothing is.
22	Receives the DTO from the controller.
23–28	Create an entity from the DTO data. <code>request.author()</code> is the record's getter.
29	Saves to PostgreSQL . Hibernate runs <code>INSERT INTO messages ...</code> (visible in the log thanks to <code>show-sql</code>). The returned object has the new <code>id</code> and <code>savedAt</code> .
30	Convert to the response DTO.
33	A read-only transaction for reading; slightly more efficient.
34–37	Read all rows (<code>SELECT * FROM messages</code>), convert each entity with <code>MessageResponse::from</code> using a stream, and collect them into a list.

9.7 The controller for messages

In `com.example.storage.controller` create a second class, `MessageController`. The Part 1 `StorageController` stays unchanged.

`storage-service/.../controller/MessageController.java`

```

1 package com.example.storage.controller;
2

```

```

3 import com.example.storage.dto.MessageResponse;
4 import com.example.storage.dto.SaveMessageRequest;
5 import com.example.storage.service.MessageService;
6 import org.springframework.http.HttpStatus;
7 import org.springframework.web.bind.annotation.GetMapping;
8 import org.springframework.web.bind.annotation.PostMapping;
9 import org.springframework.web.bind.annotation.RequestBody;
10 import org.springframework.web.bind.annotation.RequestMapping;
11 import org.springframework.web.bind.annotation.ResponseStatus;
12 import org.springframework.web.bind.annotation.RestController;
13
14 import java.util.List;
15
16 @RestController
17 @RequestMapping("/api/storage/messages")
18 public class MessageController {
19
20     private final MessageService messageService;
21
22     public MessageController(MessageService messageService) {
23         this.messageService = messageService;
24     }
25
26     @PostMapping
27     @ResponseStatus(HttpStatus.CREATED)
28     public MessageResponse save(@RequestBody SaveMessageRequest request) {
29         return messageService.save(request);
30     }
31
32     @GetMapping
33     public List<MessageResponse> findAll() {
34         return messageService.findAll();
35     }
36 }

```

Line	What it means
16	REST controller, as before.
17	Base URL for this controller. It must match the paths in the Feign client exactly.
20–24	Inject the service.
26	Handles POST <code>/api/storage/messages</code> (POST = create new data).
27	Answer with status 201 Created instead of the default 200 OK. That is the correct status when something new was created.
28	<code>@RequestBody</code> = take the JSON from the request body and convert it into a <code>SaveMessageRequest</code> .
29	Save and return the stored record as JSON.
32–35	<code>GET /api/storage/messages</code> returns all messages as a JSON array.

9.8 Test storage-service alone

1. Check that the postgres container is running (`docker compose ps`).
2. Run `StorageServiceApplication` in IntelliJ. In the log you should see `HikariPool-1 - Start completed` (database connected) and a `create table messages ...` statement from Hibernate.
3. Open `http://localhost:8082/api/storage/messages` in the browser. You get `[]`, an empty list, because nothing is saved yet.

CHECKPOINT: storage-service connects to PostgreSQL and the table `messages` has been created. Keep it running.

10. Part 2 — client-service Processes and Forwards

client-service needs **no new dependencies**: it does not talk to the database, only to storage-service.

10.1 DTOs

Create package `com.example.client.dto` with three records:

`client-service/.../dto/MessageRequest.java`

```
1 package com.example.client.dto;
2
3 public record MessageRequest(String author, String text) {
4 }
```

What **you** send to client-service: just `author` and `text`, possibly messy.

`client-service/.../dto/ProcessedMessage.java`

```
1 package com.example.client.dto;
2
3 import java.time.LocalDateTime;
4
5 public record ProcessedMessage(
6     String author,
7     String text,
8     int wordCount,
9     LocalDateTime processedAt
10 ) {
11 }
```

What client-service **sends to storage-service** after processing. Its JSON matches storage-service's `SaveMessageRequest` field for field.

`client-service/.../dto/StoredMessage.java`

```
1 package com.example.client.dto;
2
3 import java.time.LocalDateTime;
4
5 public record StoredMessage(
6     Long id,
7     String author,
8     String text,
9     int wordCount,
10    LocalDateTime processedAt,
11    LocalDateTime savedAt
12 ) {
13 }
```

What storage-service **returns**. It matches storage-service's `MessageResponse` field for field.

10.2 Extend the Feign client

Replace the content of `StorageClient.java` with this version. The Part 1 methods stay; two methods are added:

`client-service/.../feign/StorageClient.java`

```
1 package com.example.client.feign;
2
3 import com.example.client.dto.ProcessedMessage;
4 import com.example.client.dto.StoredMessage;
5 import org.springframework.cloud.openfeign.FeignClient;
```

```

6 import org.springframework.web.bind.annotation.GetMapping;
7 import org.springframework.web.bind.annotation.PathVariable;
8 import org.springframework.web.bind.annotation.PostMapping;
9 import org.springframework.web.bind.annotation.RequestBody;
10
11 import java.util.List;
12 import java.util.Map;
13
14 @FeignClient(name = "storage-service", url = "${storage.service.url}")
15 public interface StorageClient {
16
17     // ----- Part 1 -----
18     @GetMapping("/api/storage/ping")
19     Map<String, Object> ping();
20
21     @GetMapping("/api/storage/greet/{name}")
22     String greet(@PathVariable("name") String name);
23
24     // ----- Part 2 -----
25     @PostMapping("/api/storage/messages")
26     StoredMessage saveMessage(@RequestBody ProcessedMessage message);
27
28     @GetMapping("/api/storage/messages")
29     List<StoredMessage> getAllMessages();
30 }

```

Line	What it means
3–9	New imports for the DTOs and the POST annotations.
11	For the list of messages.
14	Same client as before, same URL setting.
17–22	Unchanged Part 1 methods.
25–26	Calling <code>saveMessage(processed)</code> sends POST <code>{url}/api/storage/messages</code> . <code>@RequestBody</code> means: turn the <code>ProcessedMessage</code> object into JSON and send it as the request body. The JSON answer is converted into a <code>StoredMessage</code> .
28–29	Sends GET and converts the JSON array into a <code>List<StoredMessage></code> .

10.3 The processing service

Create package `com.example.client.service` and class `MessageProcessingService`. This is where the "processing" of Part 2 happens:

`client-service/.../service/MessageProcessingService.java`

```

1 package com.example.client.service;
2
3 import com.example.client.dto.MessageRequest;
4 import com.example.client.dto.ProcessedMessage;
5 import com.example.client.dto.StoredMessage;
6 import com.example.client.feign.StorageClient;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.stereotype.Service;
9 import org.springframework.web.server.ResponseStatusException;
10
11 import java.time.LocalDateTime;
12 import java.util.List;
13
14 @Service
15 public class MessageProcessingService {
16
17     private final StorageClient storageClient;
18
19     public MessageProcessingService(StorageClient storageClient) {
20         this.storageClient = storageClient;

```

```

21     }
22
23     public StoredMessage processAndStore(MessageRequest request) {
24         validate(request);
25
26         String author = capitalize(request.author().trim());
27         String text = request.text().trim().replaceAll("\\s+", " ");
28         int wordCount = text.split(" ").length;
29
30         ProcessedMessage processed =
31             new ProcessedMessage(author, text, wordCount, LocalDateTime.now());
32
33         return storageClient.saveMessage(processed);
34     }
35
36     public List<StoredMessage> getAllMessages() {
37         return storageClient.getAllMessages();
38     }
39
40     private void validate(MessageRequest request) {
41         if (request == null || isBlank(request.author()) || isBlank(request.text())) {
42             throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
43                 "Both 'author' and 'text' are required");
44         }
45     }
46
47     private boolean isBlank(String value) {
48         return value == null || value.isBlank();
49     }
50
51     private String capitalize(String value) {
52         return value.substring(0, 1).toUpperCase() + value.substring(1).toLowerCase();
53     }
54 }

```

Line	What it means
9	An exception that Spring turns into an HTTP error response with the status we choose.
14	A Spring-managed service class.
17–21	Inject the Feign client.
23	The main method: process, then store.
24	Step 1: check the input first. If it's invalid, an exception stops the method here.
26	Step 2: <code>trim()</code> removes spaces at the start and end (" aLiCe " → "aLiCe"), <code>capitalize</code> makes it "Alice".
27	Step 3: trim the text and replace any run of whitespace (<code>\s+</code> is a <i>regular expression</i> meaning "one or more spaces, tabs or new lines") with a single space. In Java source code the backslash is written twice.
28	Step 4: split the text at spaces and count the pieces = number of words.
30–31	Step 5: build the processed object, adding the current time.
33	Step 6: the Feign call to storage-service, which saves the message in PostgreSQL. Its answer (the saved record) is returned.
36–38	Fetch all messages from storage-service through Feign.
40–44	If the body is missing, or <code>author</code> or <code>text</code> is empty, throw a <code>ResponseStatusException</code> with 400 Bad Request . The user gets a clear error and storage-service is never called with bad data.
47–49	True if the value is <code>null</code> or contains only spaces.
51–53	First letter upper case (<code>substring(0, 1)</code>), the rest lower case (<code>substring(1)</code>).

10.4 The public controller

In `com.example.client.controller` create `MessageController` (a different class from storage-service's controller with the same name; they live in different projects, so there is no conflict):

`client-service/.../controller/MessageController.java`

```
1  package com.example.client.controller;
2
3  import com.example.client.dto.MessageRequest;
4  import com.example.client.dto.StoredMessage;
5  import com.example.client.service.MessageProcessingService;
6  import org.springframework.http.HttpStatus;
7  import org.springframework.web.bind.annotation.GetMapping;
8  import org.springframework.web.bind.annotation.PostMapping;
9  import org.springframework.web.bind.annotation.RequestBody;
10 import org.springframework.web.bind.annotation.RequestMapping;
11 import org.springframework.web.bind.annotation.ResponseStatus;
12 import org.springframework.web.bind.annotation.RestController;
13
14 import java.util.List;
15
16 @RestController
17 @RequestMapping("/api/messages")
18 public class MessageController {
19
20     private final MessageProcessingService processingService;
21
22     public MessageController(MessageProcessingService processingService) {
23         this.processingService = processingService;
24     }
25
26     @PostMapping
27     @ResponseStatus(HttpStatus.CREATED)
28     public StoredMessage create(@RequestBody MessageRequest request) {
29         return processingService.processAndStore(request);
30     }
31
32     @GetMapping
33     public List<StoredMessage> getAll() {
34         return processingService.getAllMessages();
35     }
36 }
```

Line	What it means
17	Public base URL <code>/api/messages</code> .
20–24	Inject the processing service. The controller stays thin; logic lives in the service.
26	<code>POST /api/messages</code> creates a message.
27	Return 201 Created .
28	Read the user's JSON into a <code>MessageRequest</code> .
29	Process, forward via Feign, return the saved record.
32–35	<code>GET /api/messages</code> lists all stored messages.

10.5 Quick local test (optional)

With the postgres container and storage-service running, start `ClientServiceApplication` in IntelliJ and send a POST request as shown in section 11.3, just to `localhost`. Then stop both services in IntelliJ before the final Docker run.

11. Part 2 — Everything in Docker

11.1 The final docker-compose.yml

This is the complete file. Compare it with yours line by line:

microservices-lab/docker-compose.yml (final)

```
1  services:
2    postgres:
3      image: postgres:16
4      container_name: postgres-db
5      environment:
6        POSTGRES_DB: storedb
7        POSTGRES_USER: storage_user
8        POSTGRES_PASSWORD: storage_pass
9      ports:
10       - "5432:5432"
11     volumes:
12       - postgres-data:/var/lib/postgresql/data
13     healthcheck:
14       test: ["CMD-SHELL", "pg_isready -U storage_user -d storedb"]
15       interval: 5s
16       timeout: 5s
17       retries: 10
18
19     storage-service:
20       build: ./storage-service
21       container_name: storage-service
22       ports:
23         - "8082:8082"
24       environment:
25         DB_HOST: postgres
26         DB_PORT: 5432
27         DB_NAME: storedb
28         DB_USER: storage_user
29         DB_PASSWORD: storage_pass
30       depends_on:
31         postgres:
32           condition: service_healthy
33
34     client-service:
35       build: ./client-service
36       container_name: client-service
37       ports:
38         - "8081:8081"
39       environment:
40         STORAGE_SERVICE_URL: http://storage-service:8082
41       depends_on:
42         - storage-service
43
44   volumes:
45     postgres-data:
```

Line	What it means
2–17	The database service from chapter 8.
19	storage-service, now with database settings.
24–29	These variables fill the <code>\${DB_...}</code> placeholders in storage-service's <code>application.yml</code> . <code>DB_HOST: postgres</code> is the key line: inside Docker, the database is reached by its service name, not <code>localhost</code> .
30–32	Start storage-service only after the database healthcheck passes. Otherwise the app could start too early and crash because the database is not ready yet.
34–42	Unchanged from Part 1.

Line	What it means
44–45	Declares the named volume used by postgres. Docker creates it the first time.

11.2 Build and start everything

Stop everything running in IntelliJ, then in the `microservices-lab` folder run:

```
docker compose down
docker compose up --build
```

Wait until you see **Started StorageServiceApplication** and **Started ClientServiceApplication**. In another terminal, `docker compose ps` should list three containers, postgres being *healthy*.

11.3 Send new data

Use **one** of the following ways. The JSON contains deliberately messy spaces and letters so you can see the processing.

Windows PowerShell

```
Invoke-RestMethod -Method Post -Uri http://localhost:8081/api/messages `
-ContentType "application/json" `
-Body '{"author":" aLiCe ","text":" hello    microservices    world    "'}
```

The backtick ``` at the end of a line continues the command on the next line. You can also write it all on one line.

macOS / Linux terminal (or Git Bash)

```
curl -i -X POST http://localhost:8081/api/messages \
-H "Content-Type: application/json" \
-d '{"author":" aLiCe ","text":" hello    microservices    world    "'}
```

Windows Command Prompt (cmd)

```
curl -i -X POST http://localhost:8081/api/messages ^
-H "Content-Type: application/json" ^
-d "{\"author\": \" aLiCe \", \"text\": \" hello    microservices    world    \"}"
```

In cmd the `^` character continues a command on the next line, and quotes inside the JSON are escaped as `\"`.

Postman

1. Click **New** → **HTTP**. Choose method **POST** and URL `http://localhost:8081/api/messages`.
2. Open the **Body** tab → select **raw** → choose **JSON** in the dropdown on the right.
3. Paste `{"author": " aLiCe ", "text": " hello microservices world "}` and click **Send**.

Expected answer (**201 Created**):

```
{
  "id": 1,
  "author": "Alice",
  "text": "hello microservices world",
  "wordCount": 3,
  "processedAt": "2026-09-23T10:31:12.345678",
  "savedAt": "2026-09-23T10:31:12.412345"
}
```

Notice the processing: `aLiCe` became `Alice`, extra spaces are gone, words were counted, and the record got an `id` from PostgreSQL. Send two or three more messages, then open

`http://localhost:8081/api/messages` in the browser to see them all.

11.4 Look inside the database

Open a terminal inside the postgres container and use the `psql` command-line client:

```
docker exec -it postgres-db psql -U storage_user -d storagedb
```

You now see the prompt `storagedb=#`. Type these SQL commands (each ends with `;`):

```
\dt
SELECT * FROM messages;
\q
```

Command	Meaning
<code>\dt</code>	List tables. You should see <code>messages</code> .
<code>SELECT * FROM messages;</code>	Show all rows: <code>id</code> , <code>author</code> , <code>processed_at</code> , <code>saved_at</code> , <code>text</code> , <code>word_count</code> .
<code>\q</code>	Quit psql and return to your terminal.

You can do the same in Docker Desktop: *Containers* → *postgres-db* → *Exec* tab, then type `psql -U storage_user -d storagedb`.

11.5 Two more tests

Test 1: validation

Send a message without text, e.g. body `{"author":"Bob","text":"" }`. You get **400 Bad Request** and nothing is saved. The processing in client-service protects the database.

Test 2: data survives a restart

```
docker compose down
docker compose up -d
# wait ~20 seconds, then open http://localhost:8081/api/messages
```

Your messages are still there because they are stored in the `postgres-data` volume. Only `docker compose down -v` deletes them.

CHECKPOINT: Part 2 done. New data goes to client-service, is processed, passed through Feign to storage-service and saved in PostgreSQL, all running in Docker Desktop.

12. Troubleshooting

Read the **first error** in the log (scroll up), not the last lines. Spring errors often end with a long list of lines; the useful message is usually near the words **Caused by:**.

Problem / message	Likely cause and fix
'java' is not recognized / mvn: command not found	Not on the PATH. Reinstall with <i>Add to PATH</i> , or add the bin folder manually, then reopen the terminal.
IntelliJ shows red code everywhere	Maven not loaded or wrong SDK. Click <i>Load Maven Changes</i> ; check <i>Project Structure</i> → <i>SDK = 17</i> ; try <i>File</i> → <i>Invalidate Caches</i> → <i>Restart</i> .
Port 8081/8082 was already in use	Another copy is running (IntelliJ or Docker). Stop it: red square in IntelliJ, or docker compose down .
No qualifying bean of type 'StorageClient'	<code>@EnableFeignClients</code> is missing, or the Feign interface is outside the <code>com.example.client</code> package tree.
Could not resolve placeholder 'storage.service.url'	Typo or wrong indentation in client <code>application.yml</code> . <code>storage</code> , <code>service</code> , <code>url</code> must each be indented 2 more spaces.
feign.RetryableException: Connection refused	storage-service isn't running, or the URL is wrong. In Docker it must be <code>http://storage-service:8082</code> , not localhost.
feign.FeignException\$NotFound: [404]	The path in the Feign interface doesn't match storage-service's controller path. Compare them letter by letter.
Cannot connect to the Docker daemon / error during connect	Docker Desktop is not running. Start it and wait for <i>Engine running</i> .
Docker build fails at mvn with a network error	Internet/proxy problem while downloading dependencies. Retry; check your connection or company proxy.
Your code change doesn't appear in Docker	You forgot <code>--build</code> . Run docker compose up --build .
Connection to localhost:5432 refused (in Docker)	<code>DB_HOST: postgres</code> missing in <code>docker-compose.yml</code> for storage-service.
Connection to localhost:5432 refused (in IntelliJ)	The postgres container isn't running: docker compose up -d postgres .
password authentication failed for user	Password mismatch. Note: the postgres image only reads <code>POSTGRES_*</code> on the first start. If you changed them later, reset with docker compose down -v (deletes data).
Bind for 0.0.0.0:5432 failed: port is already allocated	A local PostgreSQL uses 5432. Use <code>"5433:5432"</code> (see chapter 8) or stop the local database.
package javax.persistence does not exist	Old tutorial code. In Spring Boot 3 use <code>jakarta.persistence</code> .
JSON field is null after the Feign call	Field names differ between the two services' DTOs (e.g. <code>wordcount</code> vs <code>wordCount</code>). They must be identical.
Spring Boot [...] is not compatible with this Spring Cloud release train	Version mismatch. Use Boot 3.3.x with Cloud 2023.0.x as in this guide.
YAML error: mapping values are not allowed here	Indentation or a missing space after <code>:</code> . Use spaces only; write key: value .

13. Command Cheat Sheet

Goal	Command
Build & start everything (logs visible)	<code>docker compose up --build</code>
Build & start in background	<code>docker compose up --build -d</code>
Start only the database	<code>docker compose up -d postgres</code>
See containers and status	<code>docker compose ps</code>
Follow logs of one service	<code>docker compose logs -f storage-service</code>
Restart one service after code change	<code>docker compose up --build -d client-service</code>
Stop and remove containers	<code>docker compose down</code>
... and delete database data	<code>docker compose down -v</code>
Open SQL console	<code>docker exec -it postgres-db psql -U storage_user -d storedb</code>
Build a jar locally	<code>mvn clean package -DskipTests</code> (inside a service folder)
Run a service locally without IntelliJ	<code>mvn spring-boot:run</code> (inside a service folder)
Free disk space from old images	<code>docker system prune</code> (asks before deleting)

14. Final Checklist

Part 1

- ☐ `storage-service` runs on port 8082 and answers `/api/storage/ping` and `/api/storage/greet/{name}`.
- ☐ `client-service` runs on port 8081 and has `@EnableFeignClients` and a `StorageClient` Feign interface.
- ☐ `/api/client/ping-storage` returns data that came from storage-service.
- ☐ Both services have a `Dockerfile`; `docker compose up --build` starts both.
- ☐ In Docker, client-service calls storage-service via `http://storage-service:8082`.

Part 2

- ☐ PostgreSQL runs in Docker with a volume and a healthcheck.
- ☐ storage-service has the JPA and PostgreSQL dependencies, the `Message` entity, the repository, service and controller.
- ☐ `POST /api/messages` on client-service validates and processes the data (trim, capitalize, word count, timestamp).
- ☐ The processed data is sent through Feign to `POST /api/storage/messages`.
- ☐ The data is visible with `SELECT * FROM messages;` and survives `docker compose down / up`.
- ☐ Invalid input returns 400 and is not saved.

Ideas for extra practice

- Add `GET /api/messages/{id}` through both services and return **404** when the ID doesn't exist.
- Use `spring-boot-starter-validation` with `@NotBlank` on `MessageRequest` instead of manual checks.
- Add a Feign `ErrorDecoder` so a storage-service error returns a clear message instead of a generic 500.

- Add `spring-boot-starter-actuator` and a Docker healthcheck on `/actuator/health` for each service.
- Replace `ddl-auto: update` with a Flyway migration script that creates the `messages` table.
- Add pgAdmin (image `dpage/pgadmin4`) to `docker-compose.yml` to browse the database in the browser.